every stage of the software development life cycle.

- DevOps teams use a variety of tools to automate the software delivery process, while DevSecOps teams use more specialised tools to scan code for security vulnerabilities and to automate security testing.

Here are some of the benefits of shifting from DevOps adoption to DevSecOps adoption.

Improved security: DevSecOps can help organisations identify and fix security vulnerabilities earlier in the software development process, which can help to reduce the risk of data breaches and other security incidents.

Improved compliance: DevSecOps can help organisations comply with industry regulations, such as PCI DSS and HIPAA.

Increased efficiency: DevSecOps can help to streamline the software delivery process, which can lead to faster time to market and reduced costs.

Improved developer productivity: It can help developers to be more productive by reducing the time they spend on security tasks.

Here are some steps you can take to shift from DevOps adoption to DevSecOps adoption.

Educate the team about DevSecOps: Make sure that everyone on the team understands the principles of DevSecOps and why it is important.

Identify security risks: Assess the current security posture and identify the areas where you are most vulnerable.

Implement security controls: Put in place security controls to mitigate the risks being identified.

Automate security testing: Use automated tools to scan the code for security vulnerabilities and to automate security testing.

Monitor your security posture: Continuously monitor the security posture to ensure that you are staying ahead of the latest threats.

Shifting from DevOps adoption to DevSecOps is a journey, not a destination. It takes time and effort to fully implement DevSecOps principles. However, the benefits of DevSecOps can be significant, including improved security, compliance, efficiency, and developer productivity.

Enabling GitLab AI services for the cloud

The following are the various GitLab AI services for a cloud platform.

- ***Code intelligence:*** GitLab AI can be used to provide code intelligence, which can help developers to write more secure and efficient code. For example, GitLab AI can help developers to identify potential security vulnerabilities, code smells, and inefficient code patterns.
- ***Code review:*** GitLab AI can be used to automate code review, which can help to improve the quality of code. For example, it can be used to identify potential security vulnerabilities and code smells.
- ***Security testing:*** GitLab AI can be used to automate security testing, which can help to identify and fix security vulnerabilities. For example, it can be used to scan code for known security vulnerabilities.
- ***Compliance reporting:*** GitLab AI can be used to generate compliance reports, which can help organisations to demonstrate compliance with industry regulations. For example, GitLab AI can be used to generate a report of all security vulnerabilities that have been identified in code.
- ***AI-assisted code review:*** This feature uses machine learning to identify potential security vulnerabilities and code smells in your code. It can also suggest improvements to your code, such as making it more efficient or easier to read.
- ***AI-assisted security testing:*** This feature uses machine learning to scan your code for known security vulnerabilities. It can also identify potential security vulnerabilities that are not yet known to the public.
- ***AI-assisted compliance reporting:*** This feature uses machine learning to generate compliance reports that demonstrate your organisation's compliance with industry regulations. It can also identify potential compliance gaps that need to be addressed.

Follow these steps to enable Gitlab AI services in a cloud platform.

1. To enable the AI/ML powered features setting, go to your GitLab instance and click on the 'Settings' icon. Then, click on 'General' and scroll down to the AI/ML powered features section. Check the box next to 'Enable AI/ML powered features' and click on 'Save changes'.
2. To create a Google Cloud service account, go to the Google Cloud platform console *https://console.cloud.google.com/* and click on the 'IAM & Admin' tab. Then, click on 'Service accounts' and on the 'Create service account' buttons. In the service account name field, enter a name for your service account. In the role field, select the 'AI Platform Admin' role. Click on the 'Create' button.
3. Once your service account has been created, click on the 'Download JSON key file' button. This will download a JSON file containing the credentials for your service account.
4. To create an environment variable in your GitLab project, go to your project's 'Settings' page and click on the 'Environment variables' tab. Click on the 'Add variable' button and enter these values.